

Lecture 1 Companion

Systems of Linear Equations — told as a story

Mathematical Foundations for Machine Learning · BITS Pilani

Prof. Saurabh

How to read this. The slides give you the math. This companion gives you the *intuition*. Every section starts with a scenario you can picture, an analogy you already know, or a small example you can do in your head. The math comes second — only when you’ve already *seen* what’s going on.

Read it like a story. Don’t worry if a formula looks scary at first; keep reading and the picture will catch up.

One image to keep in your head: a matrix is a *machine that moves space*. Solving $Ax = b$ is asking: *which input does that machine need to produce this output?* Hold this thought. We’ll come back to it again and again — it’s the single most useful mental picture in this whole course.

Contents

1	Why are we even talking about lines?	2
2	A small problem you can solve in your head	4
3	The three fates: zero, one, or infinity	5
4	Three is fine. Three thousand is not.	6
5	Meet the matrix — a spreadsheet of coefficients	7
5.1	Matrix arithmetic, in the time it takes to drink coffee	7
5.2	Why is matrix multiplication defined that strange way?	8
5.3	Special matrices to recognise	8
6	Pause. Stare at $Ax = b$ from three angles.	9
6.1	Window 1: each row is a rule	9
6.2	Window 2: b is a recipe of column vectors	9
6.3	Window 3: A is a machine that moves space	10
7	A matrix is a Photoshop filter for space	10

8	Determinant — how much did space stretch?	12
9	The undo button — the inverse A^{-1}	13
9.1	If A^{-1} is so beautiful, why don't we use it?	14
10	Echelon form — elimination, but for a computer	15
10.1	Three legal moves	15
10.2	The strategy: chase a staircase	15
10.3	Warm-up: the world's tiniest example	16
10.4	Walking through the slide example	16
11	Reduced row-echelon form — the cleanest version	18
12	Many answers? OK. Here is the recipe.	19
13	What echelon form actually unlocks	19
13.1	Rank: the “real” dimension after compression	20
13.2	Computing A^{-1} using the bed-making trick	20
13.3	Existence and uniqueness, in three lines	20
14	What's coming next	21
15	The whole story, in five lines	22

1 | Why are we even talking about lines?

OPEN INSTAGRAM. Scroll for ten seconds. The app just decided what to show you: which photo, which order, which ad. Behind that decision is a tower of math. Strip the tower down, and at the very bottom you find one humble object — a system of linear equations.

Same story for the GPS finding your route, the camera autofocusing on a face, ChatGPT predicting the next word, your bank's fraud alert. Every one of them, sooner or later, lands on something of the form

$$Ax = b.$$

So before we earn the right to talk about neural networks or transformers, we need to get *very* comfortable with this little equation. That's all of Lecture 1.

But wait — is the world actually linear?

Look out the window. Roads curve. Light bends around things. The stock market zigzags. Almost *nothing* in real life is a straight line.

So why does an entire branch of math, and the engine room of every ML model, obsess about linear things?

Intuition

Three honest reasons.

1. Linear things can always be solved. Not “solved if we’re lucky” — *always*. We can write a recipe, hand it to a computer, and either get the answer or learn for sure no answer exists. Most non-linear problems do not give you that guarantee. (For non-linear systems, even checking “does a solution exist?” can be a research-level question.)

2. Up close, almost everything looks linear. The Earth is round, but the floor of your room is flat. A roller-coaster track curves, but at any single instant it’s straight. A photograph of your face curves around your cheekbone, but each tiny pixel is a flat patch.

This is why calculus works. ML — gradient descent, neural-network training, optimisation — runs on this same trick: *zoom in enough, and the world is linear again*. Solve the easy linear problem. Take a small step. Zoom in again. Repeat a million times. That is literally how a neural network learns.

3. You can see it. Lines, planes, hyperplanes. Geometry you can draw on a napkin. That’s a luxury most other branches of math don’t give you.

The price you pay for simplicity

A single straight line cannot draw a circle. It cannot trace a roller-coaster’s curve. It cannot follow a stock-market chart. Linear things are simple, but simple things are limited.

So machine learning’s job is mostly to *stack* many simple linear pieces, with a sprinkle of non-linear glue (a function like ReLU or sigmoid) between them. Picture LEGO bricks: each brick is dumb and rectangular — but a million of them can build a castle. A neural network is exactly this: linear $\rightarrow$ nonlinear $\rightarrow$ linear $\rightarrow$ nonlinear $\rightarrow$... Mostly linear, with just enough bend to escape the limits of straightness.

What is a vector, really?

A vector is just an *ordered list of numbers* — and the rules for adding and scaling them. That’s it. Take a moment with that. Vectors are not mysterious arrows in some abstract space; they are lists.

Some lists you already use:

- A house in a real-estate dataset is a vector: (area, bedrooms, age, price).
- A photo is a vector: one number per pixel, hundreds of thousands of them in a long list.
- A word, in modern NLP, is a vector — the famous “embedding” — maybe 768 numbers long, learned by the model.
- Even *you*, in your bank’s risk model, are a vector: (age, income, credit score, ...).

For now, picture an arrow on a piece of paper, or a list like $(3, -2, 7)$. Whatever helps. Both pictures are correct, and a big part of getting good at linear algebra is fluently switching between the two.

2 | A small problem you can solve in your head

Imagine this

You run a small workshop that makes three kinds of furniture kits: *chairs*, *tables*, and *cabinets*. Each one needs different amounts of wood, labor, machine time, and shipping effort. Today you have exactly 9 units of wood, 8 hours of labor, 7 hours of machine time, and 11 units of shipping effort.

Question: how many of each kit should you make so that nothing is wasted — every resource used *exactly*?

THIS IS A REAL planning problem. A factory manager solves a version of it every Monday morning. Translate the scenario into math, one resource at a time. If you make x_1 chairs, x_2 tables, x_3 cabinets, then:

$$x_1 + 2x_2 + x_3 = 9 \quad (\text{use up all the wood})$$

$$2x_1 + x_2 + x_3 = 8 \quad (\text{use up all the labor})$$

$$x_1 + x_2 + 2x_3 = 7 \quad (\text{use up all the machine time})$$

$$3x_1 + x_2 + 2x_3 = 11 \quad (\text{use up all the shipping})$$

How would you solve this on paper? You'd play a small game. Take two equations, scale one of them, and subtract. A variable disappears. Repeat. That's *elimination*. Plug the answer back in to recover the others. That's *substitution*.

Intuition

Watch one move so you've seen it once. Take equation 2 minus $2 \times$ equation 1:

$$\underbrace{(2x_1 + x_2 + x_3)}_{\text{eq. 2}} - 2 \underbrace{(x_1 + 2x_2 + x_3)}_{\text{eq. 1}} = 8 - 2 \cdot 9$$

$$\implies -3x_2 - x_3 = -10.$$

The x_1 vanished — that is what “eliminate” literally means. Now I have a smaller problem. Repeat that same move on the other equations and eventually one variable is alone. The answer pops out: $x_1 = 2$, $x_2 = 3$, $x_3 = 1$.

Try it yourself

Plug $x_1 = 2$, $x_2 = 3$, $x_3 = 1$ into all four equations and check they all hold. That tiny act of plugging-back-in is something you should always do — not for marks, but to feel the answer land. Try it now: $1(2) + 2(3) + 1(1) = 9$. ✓

Quick question — four equations, three unknowns?

Yes. That's called *overdetermined*: more rules than variables. Usually that means no solution exists at all. The fact that one *does* exist here is a small lucky accident — the four constraints happen to agree with each other. Later in the lecture we'll learn how to check for this directly, just by reading the matrix — no guesswork needed.

The one thing to remember

Solving a linear system is just two moves — **eliminate** and **substitute**. Everything fancy in this lecture (matrices, Gaussian elimination, row-echelon form) is just those two moves, made automatic and made to scale.

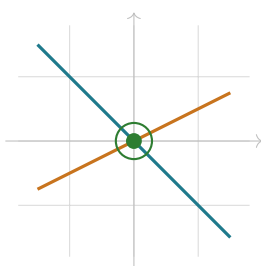


3 | The three fates: zero, one, or infinity

Imagine this

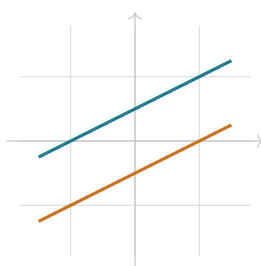
A treasure is hidden somewhere on a flat island. Two friends each give you a clue. Each clue happens to describe a straight line: “*the treasure is somewhere on this line.*” What can possibly happen when you combine their clues?

PAUSE AND TRY to enumerate the possibilities yourself before reading on. Honestly try. ...Done? You probably found three:



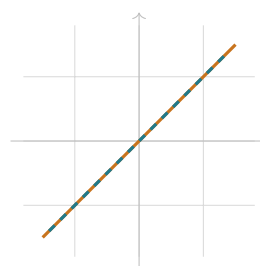
Lines cross

one X marks the spot



Parallel lines

clues disagree



Same line

whole line works

Lines cross (one point of intersection): exactly one X marks the spot.

Parallel lines: the clues never meet, so they actively disagree — nowhere works.

Same line: the second clue is just the first one rephrased; the whole line is fair game.

Aha

A linear system can never have exactly two answers. Or seven. Or forty-two. It is *zero, one, or infinity*. Those are the only options.

Why? Suppose, just suppose, you found two distinct answers x_1 and x_2 . Sketch the line connecting them: every point on that line is also a valid answer. (Half-and-half is a solution. One-third / two-thirds is a solution. And so on — you can prove this in two lines using linearity.)

So the moment you have two, you have a whole line. If you have a line in 2D, you might have a whole plane in 3D. *Finite collections of more than one solution are simply not on the menu*. The math doesn't allow it.

Higher dimensions, same story

In 3D, each equation describes a plane (a flat sheet floating in space). Three planes can:

- meet at a single point (one solution),
- all share a common line (infinitely many, lying along that line),
- all coincide as one plane (infinitely many, lying all over that plane),
- or fail to share *any* common point (no solution — maybe two are parallel, or the three pairwise intersections form a triangle).

Same three fates: zero, one, or infinity. In n dimensions we just call these surfaces *hyperplanes* and stop trying to draw them. The story doesn't change.



4 | Three is fine. Three thousand is not.

YOU HANDLED the chair-table-cabinet problem in your head. Now imagine 30 equations. Now 3000. Now a million-row regression problem with a million variables. Hand-elimination falls apart.

Intuition

Worse: when *you* eliminate by hand, you make personal decisions at every step — “hmm, let me kill x_2 first, the numbers are nicer that way.” That’s fine for one problem. Terrible for a million.

A computer can’t be moody. It needs a recipe that

- does the same kind of step at every stage (no “creative choices”),
- is guaranteed to finish (no infinite loops),
- tells you at the end which of the three fates you’re in.

That’s what an *algorithm* is. The rest of the lecture is building one.

So we need two things:

1. A **compact way to write** the system, so the computer can read it. → *matrix*.
2. A **mechanical procedure** to eliminate. → *Gaussian elimination*.

Both arrive together. The rest of the lecture is just unpacking them.



5 | Meet the matrix — a spreadsheet of coefficients

A MATRIX is a spreadsheet. Rows and columns of numbers. That’s it. If you’ve ever opened Excel, you’ve seen a matrix.

Take our furniture problem. Strip out the variable names, the plus signs, the equals signs. Keep *only* the numbers. You’re left with a grid:

$$\underbrace{\begin{bmatrix} 1 & 2 & 1 \\ 2 & 1 & 1 \\ 1 & 1 & 2 \\ 3 & 1 & 2 \end{bmatrix}}_{A \text{ (the recipe)}} \underbrace{\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}}_{\mathbf{x} \text{ (the unknowns)}} = \underbrace{\begin{bmatrix} 9 \\ 8 \\ 7 \\ 11 \end{bmatrix}}_{\mathbf{b} \text{ (the targets)}}$$

And just like that, an entire system of equations becomes three letters: $A\mathbf{x} = \mathbf{b}$.

This is more than shorthand. It’s a doorway. Once we stop seeing A as “a stack of equations” and start seeing it as “a thing that does something to $\mathbf{x}$,” the whole subject opens up.

5.1 Matrix arithmetic, in the time it takes to drink coffee

Adding two matrices: line them up and add cell by cell — only works if same shape.

Scalar multiply: multiply every cell by the number.

Multiplying two matrices is the weird one: *row-times-column*. Hold tight; in a moment we’ll see why this peculiar rule is actually beautiful.

A few facts you'll use forever:

- $(AB)C = A(BC)$ — you can regroup.
- $A(B + C) = AB + AC$ — you can distribute.
- $(AB)^T = B^T A^T$ — transpose flips the order.
- $AB \neq BA$ in general — *matrix multiplication is not commutative*. Order matters.

Intuition

Why does order matter? Think about real-life sequences. Put on socks, then shoes — fine. Reverse it (shoes first, then socks) — absurd. Same operations, different order, totally different outcome.

Or: rotate this page by 90° clockwise, then translate it 5 cm to the right. Now do the same two operations in the opposite order. The page ends up in different places. That is $AB \neq BA$.

5.2 Why is matrix multiplication defined that strange way?

The row-times-column rule looks invented and arbitrary. It is not. It is forced upon us, the moment we decide that matrices should represent transformations.

Intuition

Picture two photo filters in an editing app.

- Filter B blurs your photo.
- Filter A then darkens it.

Apply them in that order — B first, then A — and the combined effect is one new filter, “blur-and-then-darken.” As matrices, that combined filter is the product AB . (Read right-to-left: B acts first, A acts second. Yes, the order looks backwards. Yes, everyone gets confused at first. Yes, it eventually clicks.)

The row-times-column rule is exactly what makes $(AB)\mathbf{x} = A(B\mathbf{x})$ come out true — “apply AB to $\mathbf{x}$ ” equals “first apply B , then apply A .”

Matrix multiplication isn't a random recipe. It's function composition wearing a numerical disguise.

5.3 Special matrices to recognise

A few matrices show up so often they get their own names:

- **Identity** matrix I : ones on the diagonal, zeros elsewhere. The “do nothing” filter. $IA = AI = A$, always.
- **Zero** matrix: all zeros. The “destroy everything” filter.
- **Diagonal** matrix: only the diagonal is non-zero. Stretches each axis independently — think of stretching a photo's width and height by different amounts.

- **Symmetric** matrix: $A = A^T$. Mirror image across the diagonal. These are the nicest matrices in linear algebra. Lecture 4 will be very fond of them.



6 | Pause. Stare at $Ax = b$ from three angles.

THIS IS THE MOST important section of the lecture. Read it twice if you have to. We're going to look at the same equation, $Ax = b$, from three different angles. Each angle is correct. Each one answers a different question.

Switching between these three windows is the single most useful skill in linear algebra.

Anytime you're stuck on a problem, ask yourself: *which of the three windows would let me see this clearly?*

A toy example we'll use throughout

To make all three views concrete, we'll keep coming back to one tiny system:

$$\begin{array}{l} 2x_1 + x_2 = 5 \\ x_1 + 3x_2 = 5 \end{array} \iff A = \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix}, \quad b = \begin{bmatrix} 5 \\ 5 \end{bmatrix}.$$

The answer (you can verify) is $x_1 = 2, x_2 = 1$. Now let's see the same answer through three windows.

6.1 Window 1: each row is a rule

The first row of A paired with b_1 is one equation: $2x_1 + x_2 = 5$. One constraint. One rule the answer must obey. The second row is another rule: $x_1 + 3x_2 = 5$.

Geometrically, each rule is a line in 2D (a plane in 3D, a hyperplane in higher dimensions). Solving $Ax = b$ means: *find the point that obeys every rule simultaneously — the point that lies on every line / plane at once*. For our toy system, that's the single point where the two lines cross: $(2, 1)$.

Intuition

This is the schoolbook view. It's the picture you grew up with: "two lines, find where they meet." It generalises perfectly — ten equations means ten hyperplanes, find the one point in their intersection.

6.2 Window 2: b is a recipe of column vectors

Now flip your gaze. Don't look at rows; look at *columns*. Call the columns of A as $a_1, a_2, \dots, a_n$. A line of arithmetic shows that

$$Ax = x_1 a_1 + x_2 a_2 + \dots + x_n a_n.$$

For our toy system: $\mathbf{a}_1 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$ and $\mathbf{a}_2 = \begin{bmatrix} 1 \\ 3 \end{bmatrix}$, and we ask: which combination of these two columns equals $\mathbf{b} = \begin{bmatrix} 5 \\ 5 \end{bmatrix}$? The answer $(x_1, x_2) = (2, 1)$ says: “two scoops of $\mathbf{a}_1$ plus one scoop of $\mathbf{a}_2$.”

Verify: $2 \cdot \begin{bmatrix} 2 \\ 1 \end{bmatrix} + 1 \cdot \begin{bmatrix} 1 \\ 3 \end{bmatrix} = \begin{bmatrix} 5 \\ 5 \end{bmatrix}$. ✓

Intuition

$\mathbf{x}$ is a shopping list. Each column $\mathbf{a}_i$ is an ingredient. The component x_i tells you how much of ingredient i to throw in the pot. Solving $A\mathbf{x} = \mathbf{b}$ is asking: *can I mix these ingredients in some proportion to make exactly $\mathbf{b}$?*

The set of *everything* you can possibly cook up by mixing the columns (any amounts, positive or negative) is called the **column space** of A . If $\mathbf{b}$ lives inside the column space, the system has a solution. If not, no recipe works.

6.3 Window 3: A is a machine that moves space

The deepest view. A is a function. You feed it a vector $\mathbf{x}$, it spits out a vector $A\mathbf{x}$. Then $A\mathbf{x} = \mathbf{b}$ is asking: *which input does the machine need, in order to spit out $\mathbf{b}$?*

For our toy example, the machine A takes $\begin{bmatrix} 2 \\ 1 \end{bmatrix}$ as input and produces $\begin{bmatrix} 5 \\ 5 \end{bmatrix}$ as output. We knew the output. We were asked to figure out the input.

Aha

Picture an old-school vending machine. You see the chocolate bar $\mathbf{b}$ sitting in the tray. Your job: figure out which button code $\mathbf{x}$ someone punched in.

Three questions matter. Can you figure out the code at all? Could multiple codes produce the same item? Could it be that *no* code produces that item?

All three are properties of the *machine itself*, not of the equation you’re staring at. The machine’s geometry decides everything.

This is the view we’ll spend the rest of the lecture deepening. It’s also the view that 3Blue1Brown’s “Essence of Linear Algebra” series builds everything on. Highly recommended watch.



7 | A matrix is a Photoshop filter for space

HERE IS THE PICTURE you should burn into your brain. Pick a 2D matrix, say

$$A = \begin{bmatrix} 1.5 & 0.7 \\ 0.4 & 1.2 \end{bmatrix}.$$

Notice the columns: $\begin{bmatrix} 1.5 \\ 0.4 \end{bmatrix}$ and $\begin{bmatrix} 0.7 \\ 1.2 \end{bmatrix}$. Now feed A the basis vectors $\hat{i} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ and $\hat{j} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$:

$$A\hat{i} = \begin{bmatrix} 1.5 \\ 0.4 \end{bmatrix} \quad \text{and} \quad A\hat{j} = \begin{bmatrix} 0.7 \\ 1.2 \end{bmatrix}.$$

Look closely. $A\hat{i}$ is the first column of A . $A\hat{j}$ is the second column. That's not coincidence. That is the entire game.

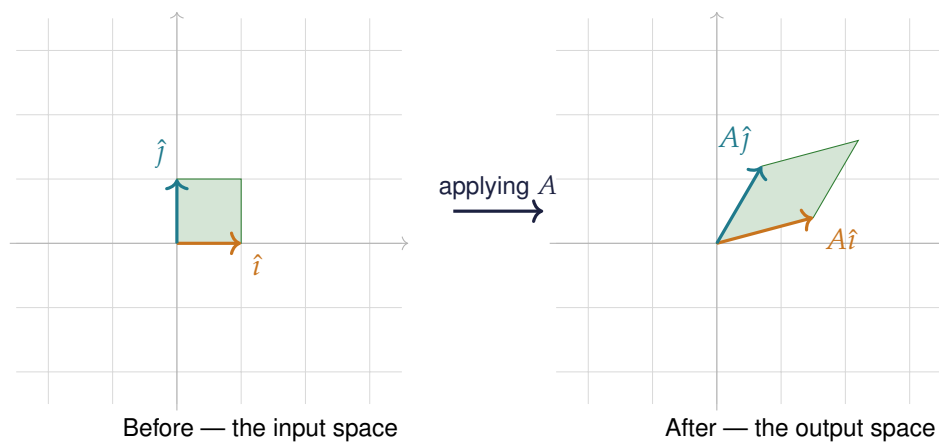
Aha

The columns of A literally tell you where $\hat{i}$ and $\hat{j}$ end up.

And once you know where $\hat{i}$ and $\hat{j}$ go, you know what A does to *every* vector. Why? Because every vector $\mathbf{x}$ is just x_1 steps along $\hat{i}$ plus x_2 steps along $\hat{j}$. Linearity then guarantees:

$$A\mathbf{x} = x_1(A\hat{i}) + x_2(A\hat{j}).$$

So a 2×2 matrix is fully described by the question “where do those two arrows land?” Four numbers, two arrows, and you’ve captured an entire transformation of the plane.



The unit square (the green tile on the left) becomes a parallelogram (the green tile on the right). Three rules survive:

- Every line stays a line (no curves appear).
- The origin stays put (the function maps $\mathbf{0} \rightarrow \mathbf{0}$).
- Parallel lines stay parallel; the grid stays a grid, just slanted.

Space has been smoothly stretched, sheared, possibly flipped — but never folded or torn. That is exactly what a linear transformation is allowed to do.

When the parallelogram is flat

Sometimes the two output arrows happen to point along the same line. The parallelogram collapses to a line segment. The whole 2D plane just got squashed onto a 1D line.

Information was destroyed. Many different inputs now share the same output — imagine Instagram applying a filter so aggressive that 100 different photos all come out looking

identical. There's no way to "undo" it. We'll meet this monster again under the name **singular matrix**.



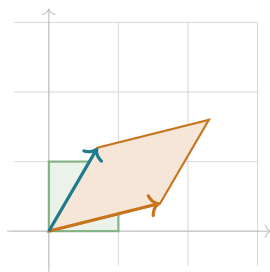
8 | Determinant — how much did space stretch?

PICK UP where we left off. The unit square has area 1. After we apply A , it becomes a parallelogram with some new area. *That new area is exactly the determinant of A .*

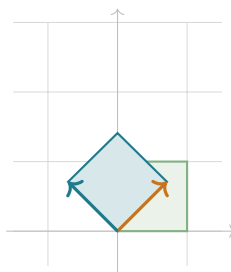
For our 2×2 matrix,

$$\det A = a_{11}a_{22} - a_{12}a_{21}.$$

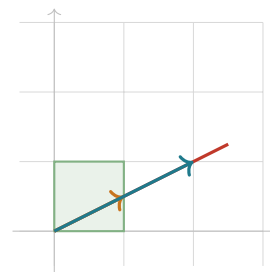
Don't memorise the formula yet — understand the picture first. The determinant tells you a single number: *by what factor did A scale areas?*



$\det A \approx 1.6$
space stretched



$\det A = 1$
pure rotation



$\det A = 0$
squashed to a line

Intuition

Determinant in plain English. The determinant carries two pieces of information at once: a magnitude and a sign.

The magnitude $|\det A|$ is the area-scaling factor.

- $|\det A| > 1$: areas grew. Space was stretched.
- $|\det A| < 1$: areas shrank, but didn't collapse.
- $|\det A| = 1$: areas unchanged.

The sign of $\det A$ tells you orientation.

- $\det A > 0$: orientation preserved. $\hat{i}$ and $\hat{j}$ are still on the same sides as before.
- $\det A < 0$: orientation flipped — the world got mirrored. (You can tell because $\hat{i}$ and $\hat{j}$ swapped sides.)

Two famous special cases: a pure rotation has $\det A = +1$ (no scaling, no flip). A pure reflection has $\det A = -1$ (no scaling, but mirrored).

And the dramatic one: $\det A = 0$. The parallelogram squashed flat to a line — or even to a single point. *A whole dimension just died.* Like juicing a 3D fruit into a 2D puddle: you cannot un-juice it.

The “ $\det = 0$ ” case is the dramatic one. It says all of these in one breath:

- A is **not invertible**. There is no undo.
- The columns of A are **linearly dependent** — one is a combination of the others, so they don't really give us anything new.
- Many directions of input space all collapse onto the same output point.

All three statements are the same fact, told in three languages. Get used to translating between them.



9 | The undo button — the inverse A^{-1}

IF A IS “translate to French”, A^{-1} is “translate back to English”. If A rotates the plane by 30° , A^{-1} rotates it back by 30° . If A stretches widths by $2\times$, A^{-1} shrinks them by $1/2$.

The matrix that exactly reverses A is called the *inverse* and written A^{-1} . Its defining property:

$$A^{-1}A = AA^{-1} = I,$$

which says: “apply A , then A^{-1} , and you're back where you started.”

If A^{-1} exists, then solving $Ax = \mathbf{b}$ is one line:

$$Ax = \mathbf{b} \implies A^{-1}Ax = A^{-1}\mathbf{b} \implies \mathbf{x} = A^{-1}\mathbf{b}.$$

For a 2×2 matrix there's even an explicit formula:

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}, \quad A^{-1} = \frac{1}{\det A} \begin{bmatrix} a_{22} & -a_{12} \\ -a_{21} & a_{11} \end{bmatrix}.$$

Notice the determinant downstairs: *if $\det A = 0$, the formula divides by zero and explodes.* That's the math version of "you can't un-juice the fruit." No undo exists. This connection (A^{-1} exists $\iff \det A \neq 0$) holds for any *square* matrix, not just 2×2 . (Non-square matrices don't have a determinant or a true inverse anyway — we'll deal with that case much later, using least squares.)

Try it yourself

For $A = \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix}$ (our toy example), $\det A = 2 \cdot 3 - 1 \cdot 1 = 5$. So

$$A^{-1} = \frac{1}{5} \begin{bmatrix} 3 & -1 \\ -1 & 2 \end{bmatrix}.$$

Multiply $A^{-1}\mathbf{b}$ with $\mathbf{b} = \begin{bmatrix} 5 \\ 5 \end{bmatrix}$ and confirm you get $\begin{bmatrix} 2 \\ 1 \end{bmatrix}$. The same answer we found before.

9.1 If A^{-1} is so beautiful, why don't we use it?

Watch out

Because computing A^{-1} explicitly is, in practice, almost always the *wrong* thing to do.

It's expensive. Roughly the same cost as solving the system directly — but with a much larger constant out front. In real ML, where A might be $10,000 \times 10,000$, that constant matters a lot.

It's fragile. If A is "almost singular" (its columns are *nearly* dependent), the inverse becomes a numerical disaster — huge entries that almost cancel, and tiny floating-point errors blow up into giant ones. Your answer is garbage and you don't know it.

It's wasteful. You only need $\mathbf{x}$ for *this* particular $\mathbf{b}$. You don't need the full matrix that solves *every* $\mathbf{b}$.

Buying a master key for an apartment building when you only want to enter one apartment, once. Beautiful in theory. Dumb in practice.

Industry rule of thumb: if you ever see `numpy.linalg.inv(A) @ b` in production code, replace it with `numpy.linalg.solve(A, b)`. Same answer, faster, more stable.

So we need a different route. Something more like the hand-elimination we already know, but turned into an algorithm a computer can follow blindly.



10 | Echelon form — elimination, but for a computer

THE TRICK is the **augmented matrix** $[A \mid \mathbf{b}]$ — glue A and $\mathbf{b}$ together into a single grid, separated by a vertical bar. Now anything you'd do to the equations in elimination has a corresponding operation on the rows of this grid.

10.1 Three legal moves

There are exactly three things you're allowed to do to rows. Each one corresponds to something obvious you'd do to the equations themselves.

1. **Swap** two rows: $R_i \leftrightarrow R_j$. (You're just changing the order in which the equations are written. The system is the same.)
2. **Scale** a row by a non-zero number λ : $R_i \rightarrow \lambda R_i$. (Multiplying both sides of an equation by the same non-zero thing.)
3. **Add** a multiple of one row to another: $R_i \rightarrow R_i + \lambda R_j$. (Adding equal things to both sides of an equation.)

Each of these is reversible. Each leaves the solution set unchanged. They only change the appearance of the system, never its meaning. We're not solving a different problem — we're rewriting the same problem until the answer becomes obvious.

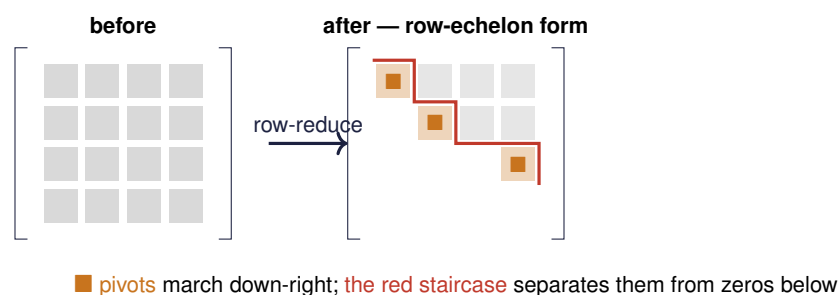
Intuition

Why must $\lambda \neq 0$ when scaling? Multiplying a row by 0 turns it into " $0 = 0$ " — a sentence that's always true, and so carries no constraint. You'd effectively be throwing away one of your equations. The remaining system can pick up extra solutions the original didn't allow. *Erasing constraints is not allowed.* Reversibility is the rule.

10.2 The strategy: chase a staircase

Use the three moves systematically. Start with column 1: get a non-zero entry at the very top (a *pivot*), then use that pivot to kill every entry below it. Move to column 2: find the next pivot below where the last one was, kill below it. Then column 3. Keep going.

The end result has the pivots marching down-and-right in a staircase pattern. We call this shape **row-echelon form (REF)**.



A matrix is in REF when (1) all-zero rows sit at the bottom, (2) each pivot is strictly to the

right of the pivot in the row above, and (3) below each pivot, the column is all zeros.

10.3 Warm-up: the world's tiniest example

Before we attempt the big slide example, let's earn our wings on something small. Solve:

$$\begin{array}{rcl} x_1 + 2x_2 = 5 \\ 3x_1 + 4x_2 = 11 \end{array} \iff \left[\begin{array}{cc|c} 1 & 2 & 5 \\ 3 & 4 & 11 \end{array} \right]$$

Step 1. Kill the 3 in row 2. Move: $R_2 \leftarrow R_2 - 3R_1$.

$$\left[\begin{array}{cc|c} 1 & 2 & 5 \\ 0 & -2 & -4 \end{array} \right]$$

That's already REF. The pivots (1 and -2) march down-right. Now read off the answer by back-substitution:

- Last row says: $-2x_2 = -4$, so $x_2 = 2$.
- First row says: $x_1 + 2(2) = 5$, so $x_1 = 1$.

That is Gaussian elimination, in its entirety. Everything we do from here is a longer version of the same dance.

10.4 Walking through the slide example

Now the bigger one. Find all solutions of (here a is a parameter we'll figure out):

$$\begin{array}{rcl} -2x_1 + 4x_2 - 2x_3 - x_4 + 4x_5 & = & -3 \\ 4x_1 - 8x_2 + 3x_3 - 3x_4 + x_5 & = & 2 \\ x_1 - 2x_2 + x_3 - x_4 + x_5 & = & 0 \\ x_1 - 2x_2 - 3x_4 + 4x_5 & = & a. \end{array}$$

Augmented matrix:

$$\left[\begin{array}{ccccc|c} -2 & 4 & -2 & -1 & 4 & -3 \\ 4 & -8 & 3 & -3 & 1 & 2 \\ 1 & -2 & 1 & -1 & 1 & 0 \\ 1 & -2 & 0 & -3 & 4 & a \end{array} \right].$$

Step 1. Swap $R_1 \leftrightarrow R_3$ to put a clean pivot on top (a 1 is friendlier than a -2):

$$\left[\begin{array}{ccccc|c} 1 & -2 & 1 & -1 & 1 & 0 \\ 4 & -8 & 3 & -3 & 1 & 2 \\ -2 & 4 & -2 & -1 & 4 & -3 \\ 1 & -2 & 0 & -3 & 4 & a \end{array} \right]$$

Step 2. Use the top pivot to kill everything below it in column 1. Three sub-moves at once: $R_2 \leftarrow R_2 - 4R_1$, $R_3 \leftarrow R_3 + 2R_1$, $R_4 \leftarrow R_4 - R_1$:

$$\left[\begin{array}{ccccc|c} 1 & -2 & 1 & -1 & 1 & 0 \\ 0 & 0 & -1 & 1 & -3 & 2 \\ 0 & 0 & 0 & -3 & 6 & -3 \\ 0 & 0 & -1 & -2 & 3 & a \end{array} \right]$$

Step 3. Move on to column 2... but wait — every entry in column 2 below row 1 is already zero. There's nothing to eliminate. (This is what a free variable looks like during the run — more on that in a moment.) Skip to column 3 instead. Use row 2 (which has -1 in column 3) as the pivot, and combine with row 3 to wipe out the -1 sitting at R_4 column 3: $R_4 \leftarrow R_4 - R_2 - R_3$:

$$\left[\begin{array}{ccccc|c} 1 & -2 & 1 & -1 & 1 & 0 \\ 0 & 0 & -1 & 1 & -3 & 2 \\ 0 & 0 & 0 & -3 & 6 & -3 \\ 0 & 0 & 0 & 0 & 0 & a+1 \end{array} \right]$$

Step 4. Tidy up. Make the pivots positive (cosmetic, but easier to read): $R_2 \leftarrow -R_2$, $R_3 \leftarrow -\frac{1}{3}R_3$:

$$\left[\begin{array}{ccccc|c} 1 & -2 & 1 & -1 & 1 & 0 \\ 0 & 0 & 1 & -1 & 3 & -2 \\ 0 & 0 & 0 & 1 & -2 & 1 \\ 0 & 0 & 0 & 0 & 0 & a+1 \end{array} \right] \quad (\text{REF}).$$

Reading the verdict from REF

Look at the last row. Every coefficient on the left-hand side is zero, and the right-hand side is $a + 1$. So the row says, simply, $0 = a + 1$. Two cases:

- If $a \neq -1$, the row reads " $0 = (\text{something non-zero})$ ". **Contradiction. No solution exists.**
- If $a = -1$, the row reads " $0 = 0$ ". Trivially true; gives us no new info, but doesn't break anything either. **Solutions exist.**

The pivot columns (columns 1, 3, 4) hold the "bound" variables. The non-pivot columns (2 and 5) hold **free variables**: free means they can take any value you like, and the bound variables adjust to compensate.

When $a = -1$, get the answer. Set the free variables (x_2 and x_5) to 0. Back-substitute, working from the bottom row up:

- Row 3 says $x_4 - 2x_5 = 1$. With $x_5 = 0$: $x_4 = 1$.
- Row 2 says $x_3 - x_4 + 3x_5 = -2$. Plug in $x_4 = 1$, $x_5 = 0$: $x_3 = -1$.
- Row 1 says $x_1 - 2x_2 + x_3 - x_4 + x_5 = 0$. Plug in everything: $x_1 + 0 - 1 - 1 + 0 = 0$, so $x_1 = 2$.

So one particular solution is $\mathbf{x}_p = (2, 0, -1, 1, 0)$.

To capture the wiggle room, we now solve the *homogeneous* system $A\mathbf{x} = \mathbf{0}$ (same REF, but with the right side replaced by zeros). For each free variable in turn, set it to 1, set the other free variable to 0, and back-substitute. Each pass gives one direction vector along which we can drift while staying a solution.

First direction. Set $x_2 = 1$, $x_5 = 0$, RHS all zero:

- Row 3: $x_4 - 2(0) = 0 \Rightarrow x_4 = 0$.
- Row 2: $x_3 - 0 + 0 = 0 \Rightarrow x_3 = 0$.

- Row 1: $x_1 - 2(1) + 0 - 0 + 0 = 0 \Rightarrow x_1 = 2$.

So $\mathbf{n}_1 = (2, 1, 0, 0, 0)$.

Second direction. Set $x_2 = 0, x_5 = 1$, RHS all zero:

- Row 3: $x_4 - 2(1) = 0 \Rightarrow x_4 = 2$.
- Row 2: $x_3 - 2 + 3(1) = 0 \Rightarrow x_3 = -1$.
- Row 1: $x_1 - 2(0) + (-1) - 2 + 1 = 0 \Rightarrow x_1 = 2$.

So $\mathbf{n}_2 = (2, 0, -1, 2, 1)$.

Total solution set: $\mathbf{x} = \mathbf{x}_p + \lambda_1 \mathbf{n}_1 + \lambda_2 \mathbf{n}_2$, for any real λ_1, λ_2 .

The one thing to remember

Echelon form turns “solve a system” into “look at a staircase”. Where the staircase ends tells you the fate (zero / one / infinitely many), and a tiny bit of back-substitution gives you the answer.



11 | Reduced row-echelon form — the cleanest version

REF IS GOOD. Reduced row-echelon form (RREF) is even better. We just keep going:

1. Scale every pivot row so the pivot itself is exactly 1.
2. Sweep *upwards* as well as downwards: kill every entry above each pivot too.

Now every pivot column is just a standard basis vector — a single 1 surrounded by zeros. The answer is sitting on the right side of the bar, ready to read.

Intuition

REF is making your bed: sheets tucked, blanket pulled up, you can sleep in it.

RREF is hospital corners: tucked, smooth, every pillow plumped, ready for inspection.

Both achieve the same thing — the same solution set — but **RREF is canonical**. Every matrix has *exactly one* RREF, no matter what order you applied your moves. It is the matrix’s fingerprint. (REF is not unique — different roads lead to different REFs of the same matrix.)



12 | Many answers? OK. Here is the recipe.

WHEN THE SYSTEM has infinitely many solutions, every one of them has the same shape:

$$\mathbf{x} = \underbrace{\mathbf{x}_p}_{\text{one valid answer}} + \underbrace{\lambda_1 \mathbf{n}_1 + \lambda_2 \mathbf{n}_2 + \dots}_{\text{wiggle room (solutions of } A\mathbf{x}=\mathbf{0})}$$

Imagine this

You walk into a Subway. The deal is: every sandwich must include bread and a protein (those are the constraints — the “ $A\mathbf{x} = \mathbf{b}$ ” part). Beyond that, you’re free to add any combination of toppings (the wiggle room).

The “particular solution” $\mathbf{x}_p$ is one specific sandwich the shop pre-makes for you: one bread, one protein, no toppings. The “free variables” are the toppings (lettuce, onion, jalapeño, sauce, ...) you choose to add. Every sandwich on the menu = the pre-made base + your topping combination.

Geometrically: the solution set is a flat (a line, or a plane, or higher), *anchored* at $\mathbf{x}_p$, free to slide around in the directions $\mathbf{n}_1, \mathbf{n}_2, \dots$. Imagine pinning a balloon to a wall with one tack: the balloon can drift in any direction parallel to the wall, but it can’t leave the wall. The tack is $\mathbf{x}_p$. The wall directions are the $\mathbf{n}_i$.

What are these direction vectors really?

They are the solutions to the simpler equation $A\mathbf{x} = \mathbf{0}$ — the inputs that A sends to zero. The set of all such inputs has a name: the **null space** of A . So:

$$\text{full solution set of } A\mathbf{x} = \mathbf{b} = \text{any one solution} + \text{null space of } A.$$

This is one of those formulas you’ll meet again in stats (linear regression), in optimisation, and in the very design of neural networks.

The one thing to remember

“How many solutions” depends only on *how much wiggle room* the matrix allows — in other words, how big its null space is. Zero wiggle room (only $\mathbf{0}$ in the null space) → one solution (or none). Some wiggle room → infinitely many.



13 | What echelon form actually unlocks

ONCE YOU KNOW how to produce REF/RREF, you’ve also unlocked most of the rest of linear algebra. Three big payoffs.

13.1 Rank: the “real” dimension after compression

Intuition

Imagine you have three friends in a group chat. Whenever one says something, the other two repeat it. There are three of them, but only *one* opinion in the room. You’d rightly say: “effectively, this is a one-person chat.”

Rank works the same way. The **rank** of A is the number of pivots in its echelon form — the number of *genuinely different* directions inside A .

If A is 3×3 but its rank is only 1, then no matter how you mix its columns, you only get a 1D line of outputs. The other two columns were “agreeing friends” — repeats (or scaled repeats) of the first.

A concrete example

Look at these two matrices:

$$B = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}, \quad C = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 10 \end{bmatrix}.$$

B has rank 2 (because $\text{row}_3 - 2\text{row}_2 + \text{row}_1 = 0$ — one row is hidden in the other two). C has rank 3 (the rows are genuinely independent). Single digit changed; rank changed; everything about how these matrices behave is now different.

The headline fact

A square $n \times n$ matrix is invertible *if and only if* its rank is the maximum value, n . Lower rank means A collapses some directions, and there is no way to undo that. This is the same statement as “ $\det A \neq 0$ ” and “columns linearly independent”. Three languages, one fact.

13.2 Computing A^{-1} using the bed-making trick

Here’s a beautiful side-effect of Gaussian elimination. Stack the identity matrix on the right of A , forming $[A \mid I]$. Now row-reduce the left half until it becomes I . Magic: the right half is now A^{-1} . (We won’t actually use this much in practice — remember the warning earlier about not computing inverses — but it’s a great proof that elimination “knows” how to invert.)

Intuition

Why does this work? Each row operation is the same as multiplying by an “elementary” matrix E_k on the left. So if a sequence $E_m \cdots E_2 E_1$ turns A into I , then that same sequence applied to I produces $E_m \cdots E_2 E_1 \cdot I = A^{-1}$. The right half is recording the inverse for free. Two for the price of one.

13.3 Existence and uniqueness, in three lines

Every question about whether a system has solutions reduces to comparing ranks:

There is at least one solution $\iff \text{rank}(A) = \text{rank}([A \mid \mathbf{b}])$.

(Translation: $\mathbf{b}$ lies in the column space of A — the columns can be mixed to produce it.)

There is exactly one solution $\iff$ above holds, *and* $\text{rank}(A)$ equals the number of variables (n).

(Translation: no free variables, so no wiggle room.)

There are infinitely many solutions $\iff$ first condition holds and $\text{rank}(A) < n$ (some variables are free).

Aha

One number — rank — answers all three questions.

That's the unreasonable power of linear algebra: a single integer summarises the entire fate of a system of equations.



14 | What's coming next

GAUSSIAN ELIMINATION is the Honda Civic of solvers. Reliable, simple, gets you everywhere, doesn't break down. For specific problems and bigger sizes there are faster, cleaner vehicles. You'll meet them in the lectures ahead.

Method	In one sentence	When
LU decomposition	Factor $A = LU$ once. Solve many $\mathbf{b}$'s cheaply. <i>Elimination, recorded.</i>	<i>Lectures 2–3</i>
Cholesky	For symmetric positive-definite A : $A = LL^T$. Half the work of LU.	<i>Lecture 4</i>
QR & least-squares	$A = QR$ where Q has orthonormal columns and R is upper-triangular. The right move when no exact solution exists. The math behind regression.	<i>Lectures 3, 7</i>
Eigen / SVD	The deepest factorisations. Reveal the matrix's stretching directions; route to PCA.	<i>Lectures 4–5</i>
Iterative methods	Conjugate gradients, GMRES. For huge sparse systems.	<i>Labs</i>

Intuition

Each of these is just elimination tuned to exploit some structure.

LU is “elimination, but write down the steps so we can reuse them on a new $\mathbf{b}$ without redoing the work.”

Cholesky is “elimination, but exploit symmetry to do half the work.”

QR is “elimination using rotations, which are kinder to round-off error.”

SVD is “elimination’s spiritual climax — factor any matrix into a stretch sandwiched between two rotations.” This one is gorgeous and we will spend a whole lecture on it.

Don’t memorise the table. Just remember: *they’re all flavours of the same elimination move you already know.*



15 | The whole story, in five lines

1. Linear systems matter because the world is locally linear, and ML is built out of stacks of linear pieces.
2. The shorthand $A\mathbf{x} = \mathbf{b}$ is really three pictures: row constraints, column recipe, transformation question. Keep all three windows open.
3. Solving directly with A^{-1} is beautiful in theory, expensive and fragile in practice. Don’t.
4. Gaussian elimination $\rightarrow$ row-echelon form $\rightarrow$ reads off solutions, inverses, rank, and the fate of the system.
5. Everything else this semester is a smarter version of elimination.

The one thing to remember

One image to keep: a matrix is a machine that moves space. Solving $A\mathbf{x} = \mathbf{b}$ is asking: *which input does this machine need to produce that output?*

One method to keep: Gaussian elimination on the augmented matrix, until you reach (R)REF.

One number to keep: rank — the matrix’s “real” dimension.

Self-check questions

If you can answer these without peeking, you’re in great shape for Lecture 2. If not, that’s OK — come back to the section listed.

1. Why must the multiplier λ in row scaling be non-zero? (§10)
2. Give a 2×2 matrix A such that $A\mathbf{x} = \mathbf{b}$ has no solution for some $\mathbf{b}$ and infinite solutions for others. (§3, §12)

3. If $A \in \mathbb{R}^{5 \times 7}$ and $\text{rank}(A) = 5$, how many solutions can $A\mathbf{x} = \mathbf{b}$ have? (§13)
4. Why is computing A^{-1} a poor way to solve a single system? (§9)
5. In your own words, what does it *mean* for a matrix to have rank 2? (§13)
6. Show, using the 2×2 formula, that A^{-1} exists iff $\det A \neq 0$, and explain $\det A = 0$ as a picture. (§8, §9)
7. In our toy system $A = \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix}$, $\mathbf{b} = \begin{bmatrix} 5 \\ 5 \end{bmatrix}$ from §6: describe the answer $\mathbf{x} = (2, 1)$ in all three windows (rule, recipe, machine). One sentence per window.

What's next

Lecture 2 zooms out: which sets of vectors are independent? Which span a plane? What's a basis? You'll meet *vector spaces* — the structure inside which every linear-algebra story lives. The matrix-as-machine intuition you built today will keep paying dividends.